

Cache



Async, Promise-based cache interface for ReactPHP.

The cache component provides a Promise-based **CacheInterface** and an in-memory **ArrayCache** implementation of that. This allows consumers to type hint against the interface and third parties to provide alternate implementations. This project is heavily inspired by PSR-16: Common Interface for Caching Libraries, but uses an interface more suited for async, non-blocking applications.

Table of Contents

- Usage
 - CacheInterface
 - * `get()`
 - * `set()`
 - * `delete()`
 - * `getMultiple()`
 - * `setMultiple()`
 - * `deleteMultiple()`
 - * `clear()`
 - * `has()`
 - ArrayCache
- Common usage
 - Fallback get
 - Fallback-get-and-set
- Install
- Tests
- License

Usage

CacheInterface

The **CacheInterface** describes the main interface of this component. This allows consumers to type hint against the interface and third parties to provide alternate implementations.

get() The `get(string $key, mixed $default = null): PromiseInterface<mixed>` method can be used to retrieve an item from the cache.

This method will resolve with the cached value on success or with the given `$default` value when no item can be found or when an error occurs. Similarly, an expired cache item (once the time-to-live is expired) is considered a cache miss.

```
$cache
    ->get('foo')
    ->then('var_dump');
```

This example fetches the value of the key `foo` and passes it to the `var_dump` function. You can use any of the composition provided by promises.

set() The `set(string $key, mixed $value, ?float $ttl = null): PromiseInterface<bool>` method can be used to store an item in the cache.

This method will resolve with `true` on success or `false` when an error occurs. If the cache implementation has to go over the network to store it, it may take a while.

The optional `$ttl` parameter sets the maximum time-to-live in seconds for this cache item. If this parameter is omitted (or `null`), the item will stay in the cache for as long as the underlying implementation supports. Trying to access an expired cache item results in a cache miss, see also `get()`.

```
$cache->set('foo', 'bar', 60);
```

This example eventually sets the value of the key `foo` to `bar`. If it already exists, it is overridden.

This interface does not enforce any particular TTL resolution, so special care may have to be taken if you rely on very high precision with millisecond accuracy or below. Cache implementations **SHOULD** work on a best effort basis and **SHOULD** provide at least second accuracy unless otherwise noted. Many existing cache implementations are known to provide microsecond or millisecond accuracy, but it's generally not recommended to rely on this high precision.

This interface suggests that cache implementations **SHOULD** use a monotonic time source if available. Given that a monotonic time source is only available as of PHP 7.3 by default, cache implementations **MAY** fall back to using wall-clock time. While this does not affect many common use cases, this is an important distinction for programs that rely on a high time precision or on systems that are subject to discontinuous time adjustments (time jumps). This means that if you store a cache item with a TTL of 30s and then adjust your system time forward by 20s, the cache item **SHOULD** still expire in 30s.

delete() The `delete(string $key): PromiseInterface<bool>` method can be used to delete an item from the cache.

This method will resolve with `true` on success or `false` when an error occurs. When no item for `$key` is found in the cache, it also resolves to `true`. If the cache implementation has to go over the network to delete it, it may take a while.

```
$cache->delete('foo');
```

This example eventually deletes the key `foo` from the cache. As with `set()`, this may not happen instantly and a promise is returned to provide guarantees whether or not the item has been removed from cache.

getMultiple() The `getMultiple(string[] $keys, mixed $default = null): PromiseInterface<array>` method can be used to retrieve multiple cache items by their unique keys.

This method will resolve with an array of cached values on success or with the given `$default` value when an item can not be found or when an error occurs. Similarly, an expired cache item (once the time-to-live is expired) is considered a cache miss.

```
$cache->getMultiple(array('name', 'age'))->then(function (array $values) {
    $name = $values['name'] ?? 'User';
    $age = $values['age'] ?? 'n/a';

    echo $name . ' is ' . $age . PHP_EOL;
});
```

This example fetches the cache items for the `name` and `age` keys and prints some example output. You can use any of the composition provided by promises.

setMultiple() The `setMultiple(array $values, ?float $ttl = null): PromiseInterface<bool>` method can be used to persist a set of key => value pairs in the cache, with an optional TTL.

This method will resolve with `true` on success or `false` when an error occurs. If the cache implementation has to go over the network to store it, it may take a while.

The optional `$ttl` parameter sets the maximum time-to-live in seconds for these cache items. If this parameter is omitted (or `null`), these items will stay in the cache for as long as the underlying implementation supports. Trying to access an expired cache items results in a cache miss, see also `getMultiple()`.

```
$cache->setMultiple(array('foo' => 1, 'bar' => 2), 60);
```

This example eventually sets the list of values - the key `foo` to 1 value and the key `bar` to 2. If some of the keys already exist, they are overridden.

deleteMultiple() The `deleteMultiple(string[] $keys): PromiseInterface<bool>` method can be used to delete multiple cache items in a single operation.

This method will resolve with `true` on success or `false` when an error occurs. When no items for `$keys` are found in the cache, it also resolves to `true`. If the cache implementation has to go over the network to delete it, it may take a while.

```
$cache->deleteMultiple(array('foo', 'bar', 'baz'));
```

This example eventually deletes keys `foo`, `bar` and `baz` from the cache. As with `setMultiple()`, this may not happen instantly and a promise is returned to provide guarantees whether or not the item has been removed from cache.

clear() The `clear(): PromiseInterface<bool>` method can be used to wipe clean the entire cache.

This method will resolve with `true` on success or `false` when an error occurs. If the cache implementation has to go over the network to delete it, it may take a while.

```
$cache->clear();
```

This example eventually deletes all keys from the cache. As with `deleteMultiple()`, this may not happen instantly and a promise is returned to provide guarantees whether or not all the items have been removed from cache.

has() The `has(string $key): PromiseInterface<bool>` method can be used to determine whether an item is present in the cache.

This method will resolve with `true` on success or `false` when no item can be found or when an error occurs. Similarly, an expired cache item (once the time-to-live is expired) is considered a cache miss.

```
$cache
->has('foo')
->then('var_dump');
```

This example checks if the value of the key `foo` is set in the cache and passes the result to the `var_dump` function. You can use any of the composition provided by promises.

NOTE: It is recommended that `has()` is only to be used for cache warming type purposes and not to be used within your live applications operations for get/set, as this method is subject to a race condition where your `has()` will return true and immediately after, another script can remove it making the state of your app out of date.

ArrayCache

The `ArrayCache` provides an in-memory implementation of the `CacheInterface`.

```
$cache = new ArrayCache();
```

```
$cache->set('foo', 'bar');
```

Its constructor accepts an optional `?int $limit` parameter to limit the maximum number of entries to store in the LRU cache. If you add more entries to this

instance, it will automatically take care of removing the one that was least recently used (LRU).

For example, this snippet will overwrite the first value and only store the last two entries:

```
$cache = new ArrayCache(2);

$cache->set('foo', '1');
$cache->set('bar', '2');
$cache->set('baz', '3');
```

This cache implementation is known to rely on wall-clock time to schedule future cache expiration times when using any version before PHP 7.3, because a monotonic time source is only available as of PHP 7.3 (`hrtime()`). While this does not affect many common use cases, this is an important distinction for programs that rely on a high time precision or on systems that are subject to discontinuous time adjustments (time jumps). This means that if you store a cache item with a TTL of 30s on PHP < 7.3 and then adjust your system time forward by 20s, the cache item may expire in 10s. See also `set()` for more details.

Common usage

Fallback get

A common use case of caches is to attempt fetching a cached value and as a fallback retrieve it from the original data source if not found. Here is an example of that:

```
$cache
    ->get('foo')
    ->then(function ($result) {
        if ($result === null) {
            return getFooFromDb();
        }

        return $result;
    })
    ->then('var_dump');
```

First an attempt is made to retrieve the value of `foo`. A callback function is registered that will call `getFooFromDb` when the resulting value is null. `getFooFromDb` is a function (can be any PHP callable) that will be called if the key does not exist in the cache.

`getFooFromDb` can handle the missing key by returning a promise for the actual value from the database (or any other data source). As a result, this chain will correctly fall back, and provide the value in both cases.

Fallback get and set

To expand on the fallback get example, often you want to set the value on the cache after fetching it from the data source.

```
$cache
->get('foo')
->then(function ($result) {
    if ($result === null) {
        return $this->getAndCacheFooFromDb();
    }

    return $result;
})
->then('var_dump');

public function getAndCacheFooFromDb()
{
    return $this->db
        ->get('foo')
        ->then(array($this, 'cacheFooFromDb'));
}

public function cacheFooFromDb($foo)
{
    $this->cache->set('foo', $foo);

    return $foo;
}
```

By using chaining you can easily conditionally cache the value if it is fetched from the database.

Install

The recommended way to install this library is through Composer. New to Composer?

This project follows SemVer. This will install the latest supported version:

```
composer require react/cache:^1.2
```

See also the CHANGELOG for details about version upgrades.

This project aims to run on any platform and thus does not require any PHP extensions and supports running on legacy PHP 5.3 through current PHP 8+ and HHVM. It's *highly recommended to use PHP 7+* for this project.

Tests

To run the test suite, you first need to clone this repo and then install all dependencies through Composer:

```
composer install
```

To run the test suite, go to the project root and run:

```
vendor/bin/phpunit
```

License

MIT, see LICENSE file.